

AI505
Optimization

Discrete Optimization
Metaheuristics

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Single Machine Total Weighted Tardiness

Given: a set of n jobs $\{1, \dots, n\}$ to be processed on a single machine and for each job j a processing time p_j , a weight w_j and a due date d_j .

Task: Find a schedule that minimizes the total weighted tardiness $\sum_{j=1}^n w_j \cdot T_j$, where $T_j = \max\{C_j - d_j, 0\}$ (C_j completion time of job j)

Example (Instance)

Job	1	2	3	4	5	6
Processing Time	3	2	2	3	4	3
Due date	6	13	4	9	7	17
Weight	2	3	1	5	1	2

Sol. representation:

Linear permutation

$$\phi = [3, 1, 5, 4, 1, 6]$$

Job	3	1	5	4	2	6
C_j	2	5	9	12	14	17
T_j	0	0	2	3	1	0
$w_j \cdot T_j$	0	0	2	15	3	0

Single Machine Total Weighted Tardiness Problem

- Interchange: size $\binom{n}{2}$ and $O(|i - j|)$ evaluation each

- first-improvement: π_j, π_k

$p_{\pi_j} \leq p_{\pi_k}$ for improvements, $w_j T_j + w_k T_k$ must decrease because jobs in $\pi_j, \dots, \pi_k$ can only increase their tardiness.

$p_{\pi_j} \geq p_{\pi_k}$ possible use of auxiliary data structure to speed up the computation

- best-improvement: π_j, π_k

$p_{\pi_j} \leq p_{\pi_k}$ for improvements, $w_j T_j + w_k T_k$ must decrease at least as the best interchange found so far because jobs in $\pi_j, \dots, \pi_k$ can only increase their tardiness.

$p_{\pi_j} \geq p_{\pi_k}$ possible use of auxiliary data structure to speed up the computation

- Swap: size $n - 1$ and $O(1)$ evaluation each
- Insert: size $(n - 1)^2$ and $O(|i - j|)$ evaluation each

But possible to speed up with systematic examination by means of swaps: an interchange is equivalent to $|i - j|$ swaps hence overall examination takes $O(n^2)$.

- The ROAR-NET API makes difficult but not impossible to implement these "efficiency tricks".

Remarks

See solutions to Sheet 11 on SMTWTP

- Logging
- Testing
- Multiple neighborhoods
- Metaheuristics

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

SAT Problem

Satisfiability problem in propositional logic

$$\begin{aligned} & (x_5 \vee x_8 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_1 \vee \bar{x}_3) \wedge (\bar{x}_8 \vee \bar{x}_3 \vee \bar{x}_7) \wedge (\bar{x}_5 \vee x_3 \vee x_8) \wedge \\ & (\bar{x}_6 \vee \bar{x}_1 \vee \bar{x}_5) \wedge (x_8 \vee \bar{x}_9 \vee x_3) \wedge (x_2 \vee x_1 \vee x_3) \wedge (\bar{x}_1 \vee x_8 \vee x_4) \wedge \\ & (\bar{x}_9 \vee \bar{x}_6 \vee x_8) \wedge (x_8 \vee x_3 \vee \bar{x}_9) \wedge (x_9 \vee \bar{x}_3 \vee x_8) \wedge (x_6 \vee \bar{x}_9 \vee x_5) \wedge \\ & (x_2 \vee \bar{x}_3 \vee \bar{x}_8) \wedge (x_8 \vee \bar{x}_6 \vee \bar{x}_3) \wedge (x_8 \vee \bar{x}_3 \vee \bar{x}_1) \wedge (\bar{x}_8 \vee x_6 \vee \bar{x}_2) \wedge \\ & (x_7 \vee x_9 \vee \bar{x}_2) \wedge (x_8 \vee \bar{x}_9 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_9 \vee x_4) \wedge (x_8 \vee x_1 \vee \bar{x}_2) \wedge \\ & (x_3 \vee \bar{x}_4 \vee \bar{x}_6) \wedge (\bar{x}_1 \vee \bar{x}_7 \vee x_5) \wedge (\bar{x}_7 \vee x_1 \vee x_6) \wedge (\bar{x}_5 \vee x_4 \vee \bar{x}_6) \wedge \\ & (\bar{x}_4 \vee x_9 \vee \bar{x}_8) \wedge (x_2 \vee x_9 \vee x_1) \wedge (x_5 \vee \bar{x}_7 \vee x_1) \wedge (\bar{x}_7 \vee \bar{x}_9 \vee \bar{x}_6) \wedge \\ & (x_2 \vee x_5 \vee x_4) \wedge (x_8 \vee \bar{x}_4 \vee x_5) \wedge (x_5 \vee x_9 \vee x_3) \wedge (\bar{x}_5 \vee \bar{x}_7 \vee x_9) \wedge \\ & (x_2 \vee \bar{x}_8 \vee x_1) \wedge (\bar{x}_7 \vee x_1 \vee x_5) \wedge (x_1 \vee x_4 \vee x_3) \wedge (x_1 \vee \bar{x}_9 \vee \bar{x}_4) \wedge \\ & (x_3 \vee x_5 \vee x_6) \wedge (\bar{x}_6 \vee x_3 \vee \bar{x}_9) \wedge (\bar{x}_7 \vee x_5 \vee x_9) \wedge (x_7 \vee \bar{x}_5 \vee \bar{x}_2) \wedge \\ & (x_4 \vee x_7 \vee x_3) \wedge (x_4 \vee \bar{x}_9 \vee \bar{x}_7) \wedge (x_5 \vee \bar{x}_1 \vee x_7) \wedge (x_5 \vee \bar{x}_1 \vee x_7) \wedge \\ & (x_6 \vee x_7 \vee \bar{x}_3) \wedge (\bar{x}_8 \vee \bar{x}_6 \vee \bar{x}_7) \wedge (x_6 \vee x_2 \vee x_3) \wedge (\bar{x}_8 \vee x_2 \vee x_5) \end{aligned}$$

Does there exist a truth assignment satisfying all clauses?

Search for a satisfying assignment (or prove none exists)

SAT Problem

Satisfiability problem in propositional logic

$$\begin{aligned} & (x_5 \vee x_8 \vee \bar{x}_2) \wedge (x_2 \vee \bar{x}_1 \vee \bar{x}_3) \wedge (\bar{x}_8 \vee \bar{x}_3 \vee \bar{x}_7) \wedge (\bar{x}_5 \vee x_3 \vee x_8) \wedge \\ & (\bar{x}_6 \vee \bar{x}_1 \vee \bar{x}_5) \wedge (x_8 \vee \bar{x}_9 \vee x_3) \wedge (x_2 \vee x_1 \vee x_3) \wedge (\bar{x}_1 \vee x_8 \vee x_4) \wedge \\ & (\bar{x}_9 \vee \bar{x}_6 \vee x_8) \wedge (x_8 \vee x_3 \vee \bar{x}_9) \wedge (x_9 \vee \bar{x}_3 \vee x_8) \wedge (x_6 \vee \bar{x}_9 \vee x_5) \wedge \\ & (x_2 \vee \bar{x}_3 \vee \bar{x}_8) \wedge (x_8 \vee \bar{x}_6 \vee \bar{x}_3) \wedge (x_8 \vee \bar{x}_3 \vee \bar{x}_1) \wedge (\bar{x}_8 \vee x_6 \vee \bar{x}_2) \wedge \\ & (x_7 \vee x_9 \vee \bar{x}_2) \wedge (x_8 \vee \bar{x}_9 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_9 \vee x_4) \wedge (x_8 \vee x_1 \vee \bar{x}_2) \wedge \\ & (x_3 \vee \bar{x}_4 \vee \bar{x}_6) \wedge (\bar{x}_1 \vee \bar{x}_7 \vee x_5) \wedge (\bar{x}_7 \vee x_1 \vee x_6) \wedge (\bar{x}_5 \vee x_4 \vee \bar{x}_6) \wedge \\ & (\bar{x}_4 \vee x_9 \vee \bar{x}_8) \wedge (x_2 \vee x_9 \vee x_1) \wedge (x_5 \vee \bar{x}_7 \vee x_1) \wedge (\bar{x}_7 \vee \bar{x}_9 \vee \bar{x}_6) \wedge \\ & (x_2 \vee x_5 \vee x_4) \wedge (x_8 \vee \bar{x}_4 \vee x_5) \wedge (x_5 \vee x_9 \vee x_3) \wedge (\bar{x}_5 \vee \bar{x}_7 \vee x_9) \wedge \\ & (x_2 \vee \bar{x}_8 \vee x_1) \wedge (\bar{x}_7 \vee x_1 \vee x_5) \wedge (x_1 \vee x_4 \vee x_3) \wedge (x_1 \vee \bar{x}_9 \vee \bar{x}_4) \wedge \\ & (x_3 \vee x_5 \vee x_6) \wedge (\bar{x}_6 \vee x_3 \vee \bar{x}_9) \wedge (\bar{x}_7 \vee x_5 \vee x_9) \wedge (x_7 \vee \bar{x}_5 \vee \bar{x}_2) \wedge \\ & (x_4 \vee x_7 \vee x_3) \wedge (x_4 \vee \bar{x}_9 \vee \bar{x}_7) \wedge (x_5 \vee \bar{x}_1 \vee x_7) \wedge (x_5 \vee \bar{x}_1 \vee x_7) \wedge \\ & (x_6 \vee x_7 \vee \bar{x}_3) \wedge (\bar{x}_8 \vee \bar{x}_6 \vee \bar{x}_7) \wedge (x_6 \vee x_2 \vee x_3) \wedge (\bar{x}_8 \vee x_2 \vee x_5) \end{aligned}$$

Does there exist a truth assignment satisfying all clauses?

Search for a satisfying assignment (or prove none exists)

Motivation

- SAT used to solve many other problems!
- Applications:
Hardware and Software Verification, Planning, Scheduling, Optimal Control, Protocol Design, Routing, Combinatorial problems, Equivalence Checking, etc.
- From 100 variables, 200 constraints (early 90s)
to 1,000,000 vars. and 20,000,000 clauses in 20 years.

Propositional logic: Syntax

Propositional logic is the simplest logic—illustrates basic ideas

There are other types of logic: first-order logic, temporal logic, etc.

The proposition symbols x_1 , x_2 , etc. are sentences

If x is a sentence, $\neg x$ is a sentence (negation)

If x_1 and x_2 are sentences, $x_1 \wedge x_2$ is a sentence (conjunction)

If x_1 and x_2 are sentences, $x_1 \vee x_2$ is a sentence (disjunction)

If x_1 and x_2 are sentences, $x_1 \implies x_2$ is a sentence (implication)

If x_1 and x_2 are sentences, $x_1 \equiv x_2$ is a sentence (biconditional)

Logical equivalence

Two sentences are logically equivalent iff true in same models: $\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$

$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$

$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$

$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$

$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$

$$(\alpha \implies \beta) \equiv (\neg\beta \implies \neg\alpha) \quad \text{contraposition}$$

$$(\alpha \implies \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination}$$

$$(\alpha \equiv \beta) \equiv ((\alpha \implies \beta) \wedge (\beta \implies \alpha)) \quad \text{bicond. elimination}$$

$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{De Morgan}$$

$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{De Morgan}$$

$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$

$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

Validity and Satisfiability

A sentence is valid if it is true in **all** models,

e.g., *True*, $A \vee \neg A$, $A \implies A$, $(A \wedge (A \implies B)) \implies B$

A sentence is satisfiable if it is true in **some** model

e.g., $A \vee B$, C

A sentence is unsatisfiable if it is true in **no** models

e.g., $A \wedge \neg A$

Conjunctive Normal Form

Every sentence in Propositional Logic is logically equivalent to a conjunction of clauses:

- A formula is in **conjunctive normal form (CNF)** iff it is of the form

$$\bigwedge_{i=1}^m \bigvee_{j=1}^{k_i} l_{ij} = (l_{11} \vee \dots \vee l_{1k_1}) \wedge \dots \wedge (l_{m1} \vee \dots \vee l_{mk_m})$$

where each **literal** l_{ij} is a propositional variable or its negation.

The disjunctions of literals: $c_i = (l_{i1} \vee \dots \vee l_{ik_i})$ are called **clauses**.

- A formula is in **k -CNF** iff it is in CNF and all clauses contain exactly k literals (*i.e.*, for all i , $k_i = k$).
- In many cases, the restriction of SAT to CNF formulae is considered.
- For every propositional formula, there is an equivalent formula in 3-CNF.

Example:

$$F := \bigwedge (\neg x_2 \vee x_1)$$

$$\bigwedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

$$\bigwedge (x_1 \vee x_2)$$

$$\bigwedge (\neg x_4 \vee x_3)$$

$$\bigwedge (\neg x_5 \vee x_3)$$

- F is in CNF.
- Is F satisfiable?

Yes, e.g., $x_1 := x_2 := \top$, $x_3 := x_4 := x_5 := \perp$ is a model of F .

Conversion to CNF

$$x_1 \iff (x_2 \vee x_3)$$

1. Eliminate $\iff$, replacing $\alpha \iff \beta$ with $(\alpha \implies \beta) \wedge (\beta \implies \alpha)$.

$$(x_1 \implies (x_2 \vee x_3)) \wedge ((x_2 \vee x_3) \implies x_1)$$

2. Eliminate $\implies$, replacing $\alpha \implies \beta$ with $\neg\alpha \vee \beta$.

$$(\neg x_1 \vee x_2 \vee x_3) \wedge (\neg(x_2 \vee x_3) \vee x_1)$$

3. Move $\neg$ inwards using de Morgan's rules and double-negation:

$$(\neg x_1 \vee x_2 \vee x_3) \wedge ((\neg x_2 \wedge \neg x_3) \vee x_1)$$

4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:

$$(\neg x_1 \vee x_2 \vee x_3) \wedge (\neg x_2 \vee x_1) \wedge (\neg x_3 \vee x_1)$$

SAT Problem: The General Problem

SAT Problem (decision problem, search variant):

- **Given:** Formula F in propositional logic
- **Task:** Find an assignment of truth values to variables in F that renders F true, or decide that no such assignment exists.

SAT Problem: A Specific Problem Instance

- **Given:** Formula $F := (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$
- **Task:** Find an assignment of truth values to variables x_1, x_2 that renders F true, or decide that no such assignment exists.

Max SAT

Definition ((MAXIMUM) k -SATISFIABILITY (SAT))

Input: A set X of variables, a collection C of disjunctive clauses of at most k literals, where a literal is a variable or a negated variable in X .

k is a constant, $k > 2$.

Task: A truth assignment for X or a truth assignment that maximizes the number of clauses satisfied.

MAX-SAT (optimization problem)

Which is the maximal number of clauses satisfiable in a propositional logic formula F ?

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Escaping Local Optima

Possibilities:

- **Restart:** re-initialize search whenever a local optimum is encountered.
(Often rather ineffective due to cost of initialization.)
- **Non-improving steps:** in local optima, allow selection of candidate solutions with equal or worse evaluation function value, *e.g.*, using minimally worsening steps.
(Can lead to long walks in **plateaus**, *i.e.*, regions of search positions with identical evaluation function.)
- **Diversify the neighborhood**

Note: None of these mechanisms is guaranteed to always escape effectively from local optima.

A Trade Off

Diversification vs Intensification

- Goal-directed and randomized components of LS strategy need to be balanced carefully.
- **Intensification (aka exploitation)**: aims at greedily increasing solution quality, *e.g.*, by exploiting the evaluation function.
- **Diversification (aka exploration)**: aims at preventing search stagnation, that is, the search process getting trapped in confined regions.

Examples:

- Iterative Improvement (First improvement or Best improvement): **intensification** strategy.
- Uninformed Random Walk/Picking (URW/P): **diversification** strategy.

Balanced combination of intensification and diversification mechanisms forms the basis for advanced LS methods.

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Greedy Local Search

Key idea: Best improvement (Hill Climber or Steepest Descent) + Sideways Moves + seldom worsening moves

Algorithm 6.1: GSAT (F)

```
Input      : A CNF formula  $F$ 
Parameters : Integers MAX-FLIPS, MAX-TRIES
Output     : A satisfying assignment for  $F$ , or FAIL
begin
  for  $i \leftarrow 1$  to MAX-TRIES do
     $\sigma \leftarrow$  a randomly generated truth assignment for  $F$ 
    for  $j \leftarrow 1$  to MAX-FLIPS do
      if  $\sigma$  satisfies  $F$  then return  $\sigma$                                 // success
       $v \leftarrow$  a variable flipping which results in the greatest decrease
                        (possibly negative) in the number of unsatisfied clauses
      Flip  $v$  in  $\sigma$ 
    return FAIL                                                            // no satisfying assignment found
end
```

- GSAT begins with a rapid greedy descent towards a better truth assignment
- then long sequences of **sideways** moves take place. Sideways moves are moves that do not increase or decrease the total number of unsatisfied clauses. They navigate through **plateaux**, which is SAT are many and large
- GSAT [Selman et al. 1992] at its times was able to beat complete search algorithms (they were not using CDC)

Randomized Iterative Improvement

aka, Stochastic Hill Climbing

Key idea: Allow worsening moves: In each search step, with a fixed probability perform an uninformed random walk step instead of an iterative improvement step.
greedy + uniform random walk

Randomized Iterative Improvement (RII):

determine initial candidate solution s

while termination condition is not satisfied **do**

 With probability w_p :

 choose a neighbor s' of s uniformly at random

 Otherwise:

 choose a neighbor s' of s such that $f(s') < f(s)$ or,

 if no such s' exists, choose s' such that $f(s')$ is minimal

$s := s'$

With infinite time it reaches optimum with $p > 0$ [Hoos and Tsang].

Example: Randomized Iterative Improvement for SAT

```
procedure RIISAT( $F$ ,  $w_p$ ,  $\text{maxSteps}$ )  
  input: a formula  $F$ , probability  $w_p$ , integer  $\text{maxSteps}$   
  output: a model  $\varphi$  for  $F$  or  $\emptyset$   
  
  choose assignment  $\varphi$  for  $F$  uniformly at random;  
  steps := 0;  
  while not( $\varphi$  is not proper) and (steps < maxSteps) do  
    with probability  $w_p$  do  
      select  $x$  in  $X$  uniformly at random and flip;  
    otherwise  
      select  $x$  in  $X^c$  uniformly at random from those that  
        maximally decrease number of clauses violated;  
      change  $\varphi$ ;  
      steps := steps + 1;  
    end  
    if  $\varphi$  is a model for  $F$  then return  $\varphi$   
    else return  $\emptyset$   
  end  
end RIISAT
```

X^c set of variables in violated clauses

Note:

- No need to terminate search when local minimum is encountered

Instead: Impose limit on number of search steps or CPU time, from beginning of search or after last improvement.

- Probabilistic mechanism permits arbitrary long sequences of random walk steps

Therefore: When run sufficiently long, RII is guaranteed to find (optimal) solution to any problem instance with arbitrarily high probability.

- GWSAT [Selman et al., 1994], was at some point state-of-the-art for SAT.

Focused Local Search: WalkSAT

```
procedure WalkSAT ( $F$ ,  $maxTries$ ,  $maxSteps$ ,  $slc$ )  
  input: CNF formula  $F$ , positive integers  $maxTries$  and  $maxSteps$ ,  
    heuristic function  $slc$   
  output: model of  $F$  or 'no solution found'  
  for  $try := 1$  to  $maxTries$  do  
     $a :=$  randomly chosen assignment of the variables in formula  $F$ ;  
    for  $step := 1$  to  $maxSteps$  do  
      if  $a$  satisfies  $F$  then return  $a$  end  
       $c :=$  randomly selected clause unsatisfied under  $a$ ;  
       $x :=$  variable selected from  $c$  according to heuristic function  $slc$ ;  
       $a := a$  with  $x$  flipped;  
    end  
  end  
  return 'no solution found'  
end WalkSAT
```

Example of **slc** heuristic: with prob. wp select a random move, with prob. $1 - wp$ select the best

Extension to CSP. Recall the Definitions

Constraint Satisfaction Problem (CSP)

A CSP is a finite set of variables X , together with a finite set of constraints C , each on a subset of X . A **solution** to a CSP is an assignment of a value $d \in D(x)$ to each $x \in X$, such that all constraints are satisfied simultaneously.

Constraint Optimization Problem (COP)

A COP is a CSP P defined on the variables $x_1, \dots, x_n$, together with an objective function $f : D(x_1) \times \dots \times D(x_n) \rightarrow Q$ that assigns a value to each assignment of values to the variables. An **optimal solution** to a minimization (maximization) COP is a solution d to P that minimizes (maximizes) the value of $f(d)$.

Recall: Constraints in a CSP can be relaxed and their violations determine the evaluation function. This is the most common approach in LS

Min-Conflict Heuristic

procedure *MCH* (*P*, *maxSteps*)

input: *CSP instance P, positive integer maxSteps*

output: *solution of P or “no solution found”*

a := randomly chosen assignment of the variables in *P*;

for *step* := 1 **to** *maxSteps* **do**

if *a* satisfies all constraints of *P* **then return** *a* **end**

x := randomly selected variable from conflict set $K(a)$;

v := randomly selected value from the domain of *x* such that
 setting *x* to *v* minimises the number of unsatisfied constraints;

a := *a* with *x* set to *v*;

end

return “no solution found”

end *MCH*

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search**

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Guided Local Search

- **Key Idea:** Modify the evaluation function whenever a local optimum is encountered.
- Associate **weights (penalties)** with solution components; these determine impact of components on evaluation function value.
- Perform Iterative Improvement; when in local minimum, increase penalties of some solution components until improving steps become available.

Guided Local Search (GLS):

determine **initial candidate solution** s

initialize penalties

while **termination criterion** is not satisfied **do**

 compute **modified evaluation function** g' from g
 based on **penalties**

 perform **subsidiary local search** on s
 using **evaluation function** g'

update penalties based on s

Guided Local Search

- **Modified evaluation function:**

$$g'(s) := f(s) + \sum_{i \in SC(s)} \text{penalty}(i),$$

where $SC(s)$ is the set of solution components used in candidate solution s .

- **Penalty initialization:** For all i : $\text{penalty}(i) := 0$.
- **Penalty update** in local minimum s : Typically involves **penalty increase** of some or all solution components of s ; often also occasional **penalty decrease** or **penalty smoothing**.
- **Subsidiary local search:** Often **Iterative Improvement**.

Potential problem:

Solution components required for (optimal) solution may also be present in many local minima.

Possible solutions:

A: Occasional decreases/smoothing of penalties.

B: Only increase penalties of solution components that are least likely to occur in (optimal) solutions.

Implementation of **B**: Only increase penalties of solution components i with maximal utility [Voudouris and Tsang, 1995]:

$$\text{util}(s, i) := \frac{f_i(s)}{1 + \text{penalty}(i)}$$

where $f_i(s)$ is the solution quality contribution of i in s .

Example: Guided Local Search (GLS) for the TSP

[Voudouris and Tsang 1995; 1999]

- **Given:** TSP instance π
- **Search space:** Hamiltonian cycles in π with n vertices;
- **Neighborhood:** 2-edge-exchange;
- **Solution components** edges of π ;
 $f_e(G, p) := w(e)$;
- **Penalty initialization:** Set all edge penalties to zero.
- **Subsidiary local search:** Iterative First Improvement.
- **Penalty update:** Increment penalties of all edges with maximal utility by

$$\lambda := 0.3 \cdot \frac{w(s_{2-opt})}{n}$$

where s_{2-opt} = 2-optimal tour.

Guided Local Search for SAT

- Assign a positive weight to each clause
- attempt to minimize the sum of the weights of the unsatisfied clauses.
- The clause weights are dynamically modified (additively or multiplicatively) as the search progresses, increasing the weight of the clauses that are currently unsatisfied.
- Depends on:
 - how often and by how much the weights of unsatisfied clauses are increased, and
 - how are all weights periodically decreased in order to prevent certain weights from becoming dis-proportionately high.

Discrete Lagrangian Method

- Change the objective function bringing constraints g_i into it

$$L(\mathbf{s}, \boldsymbol{\lambda}) = f(\mathbf{s}) + \sum_i \lambda_i g_i(\mathbf{s})$$

- λ_i are continuous variables called Lagrangian Multipliers
- $L(\mathbf{s}^*, \boldsymbol{\lambda}) \leq L(\mathbf{s}^*, \boldsymbol{\lambda}^*) \leq L(\mathbf{s}, \boldsymbol{\lambda}^*)$
- Alternate optimizations in $\mathbf{s}$ and in $\boldsymbol{\lambda}$

Discrete Lagrangian Method for SAT

let $U_i(\mathbf{x})$ be a function that is 0 if C_i is satisfied by a solution $\mathbf{x}$, and 1 otherwise.

$$\begin{aligned} \text{minimize } N(\mathbf{x}) &= \sum_{i=1}^m U_i(\mathbf{x}) \\ \text{s.t. } U_i(\mathbf{x}) &= 0 \quad \forall i \in \{1, 2, \dots, m\} \end{aligned}$$

Discrete Lagrangian Function:

$$L_d(\mathbf{x}, \lambda) = N(\mathbf{x}) + \sum_{i=1}^m \lambda_i U_i(\mathbf{x})$$

A point $(\mathbf{x}^*, \lambda^*) \in \{0, 1\}^n \times \mathbb{R}^m$ is called a **saddle point** of the Lagrange function $L_d(\mathbf{x}, \lambda)$ if it is a local minimum w.r.t. $\mathbf{x}^*$ and a local maximum w.r.t. λ^* . Formally, $(\mathbf{x}^*, \lambda^*)$ is a saddle point for L_d if $L_d(\mathbf{x}^*, \lambda) \leq L_d(\mathbf{x}^*, \lambda^*) \leq L_d(\mathbf{x}, \lambda^*)$

Probabilistic Iterative Improvement

Key idea: Accept worsening steps with probability that depends on respective deterioration in evaluation function value:
bigger deterioration $\cong$ smaller probability

Realization:

- Function $p(f, s, s')$: determines probability distribution over neighbors s' of s based on their values under evaluation function f .
- Accept s' neighbor of s with probability $p(f, s, s')$.

Note:

- Behavior of PII crucially depends on choice of p .
- II and RII are special cases of PII.

Example: Metropolis PII for the TSP

- **Search space** S : set of all Hamiltonian cycles in given graph G .
- **Solution set**: same as S
- **Neighborhood relation** $\mathcal{N}(s)$: 2-edge-exchange
- **Initialization**: an Hamiltonian cycle uniformly at random.
- **Step function**: implemented as 2-stage process:
 1. select neighbor $s' \in \mathcal{N}(s)$ uniformly at random;
 2. accept as new search position with probability:

$$p(T, s, s') := \begin{cases} 1 & \text{if } f(s') \leq f(s) \\ \exp \frac{-(f(s')-f(s))}{T} & \text{otherwise} \end{cases}$$

(Metropolis condition), where **temperature** parameter T controls likelihood of accepting worsening steps.

- **Termination**: upon exceeding given bound on run-time.

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing**

- Iterated Local Search

- Tabu Search

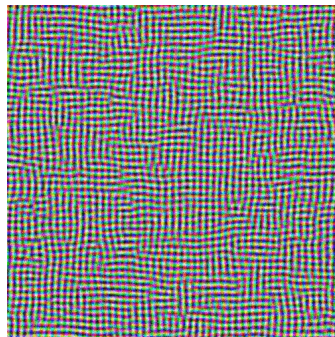
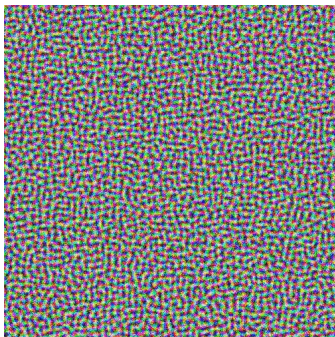
- Variable Neighborhood Search

3. Population Based Metaheuristics

Inspired by statistical mechanics in matter physics:

- candidate solutions $\cong$ states of physical system
- evaluation function $\cong$ thermodynamic energy
- globally optimal solutions $\cong$ ground states
- parameter $T \cong$ physical temperature

Note: In physical process (e.g., annealing of metals), perfect ground states are achieved by very slow lowering of temperature.



Simulated Annealing

Key idea: Vary temperature parameter, *i.e.*, probability of accepting worsening moves, in Probabilistic Iterative Improvement according to **annealing schedule** (aka **cooling schedule**).

Simulated Annealing (SA):

determine initial candidate solution s

set initial temperature T according to **annealing schedule**

while termination condition is not satisfied: **do**

while maintain same temperature T according to **annealing schedule** **do**

 probabilistically choose a neighbor s' of s using **proposal mechanism**

if s' satisfies probabilistic **acceptance criterion** (depending on T) **then**

$s := s'$

 update T according to **annealing schedule**

- 2-stage step function based on
 - proposal mechanism (often uniform random choice from $N(s)$)
 - acceptance criterion (often **Metropolis condition**)
- Annealing schedule
(function mapping run-time t onto temperature $T(t)$):
 - initial temperature T_0
(may depend on properties of given problem instance)
 - temperature update scheme
(e.g., linear cooling: $T_{i+1} = T_0(1 - i/I_{max})$,
geometric cooling: $T_{i+1} = \alpha \cdot T_i$)
 - number of search steps to be performed at each temperature
(often multiple of neighborhood size)
 - may be **static** or **dynamic**
 - seek to balance moderate execution time with asymptotic behavior properties
- Termination predicate: often based on **acceptance ratio**,
i.e., ratio accepted / proposed steps **or** number of idle iterations

Example: Simulated Annealing for TSP

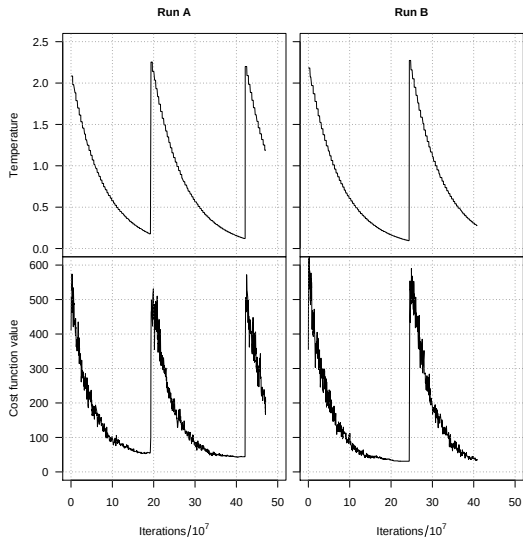
Extension of previous PII algorithm for the TSP, with

- **proposal mechanism:** uniform random choice from 2-exchange neighborhood;
- **acceptance criterion:** Metropolis condition (always accept improving steps, accept worsening steps with probability $\exp[-(f(s') - f(s))/T]$);
- **annealing schedule:** geometric cooling $T := 0.95 \cdot T$ with $n \cdot (n - 1)$ steps at each temperature (n = number of vertices in given graph), T_0 chosen such that 97% of proposed steps are accepted;
- **termination:** when for five successive temperature values no improvement in solution quality and acceptance ratio $< 2\%$.

Improvements:

- neighborhood pruning (e.g., candidate lists for TSP)
- greedy initialization (e.g., by using NNH for the TSP)
- **low temperature starts** (to prevent good initial candidate solutions from being too easily destroyed by worsening steps)

Profiling



Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search**

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Iterated Local Search

Key Idea: Use two types of LS steps:

- **subsidiary local search** steps for reaching local optima as efficiently as possible (intensification)
- **perturbation steps** for effectively escaping from local optima (diversification).

Also: Use **acceptance criterion** to control diversification **vs** intensification behavior.

Iterated Local Search (ILS):

determine initial candidate solution s

perform **subsidiary local search** on s

while termination criterion is not satisfied **do**

$r := s$

 perform **perturbation** on s

 perform **subsidiary local search** on s

 based on **acceptance criterion**,

 keep s or revert to $s := r$

Note:

- **Subsidiary local search** results in a local minimum.
- ILS trajectories can be seen as walks in the space of local minima of the given evaluation function.
- **Perturbation phase** and **acceptance criterion** may use aspects of **search history** (*i.e.*, limited memory).
- In a high-performance ILS algorithm, **subsidiary local search**, **perturbation mechanism** and **acceptance criterion** need to complement each other well.

Components

Subsidiary local search: (1)

- More effective subsidiary local search procedures lead to better ILS performance.
Example: 2-opt **vs** 3-opt **vs** LK for TSP.
- Often, subsidiary local search = iterative improvement, but more sophisticated LS methods can be used. (e.g., Tabu Search).

Components

Perturbation mechanism: (1)

- Needs to be chosen such that its effect **cannot** be easily undone by subsequent local search phase.
(Often achieved by search steps larger neighborhood.)
Example: local search = 3-opt, perturbation = 4-exchange steps in ILS for TSP.
- A perturbation phase may consist of one or more perturbation steps.
- Weak perturbation $\Rightarrow$ short subsequent local search phase;
but: risk of revisiting current local minimum.
- Strong perturbation $\Rightarrow$ more effective escape from local minima;
but: may have similar drawbacks as random restart.
- Advanced ILS algorithms may change nature and/or strength of perturbation adaptively during search.

Components

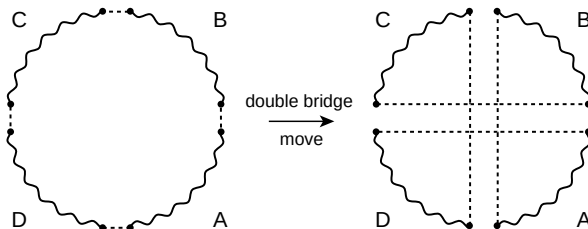
Acceptance criteria: (1)

- Always accept the **best** of the two candidate solutions
⇒ ILS performs Iterative Improvement in the space of local optima reached by subsidiary local search.
- Always accept the **most recent** of the two candidate solutions
⇒ ILS performs random walk in the space of local optima reached by subsidiary local search.
- Intermediate behavior: select between the two candidate solutions based on the **Metropolis criterion** (e.g., used in **Large Step Markov Chains** [Martin et al., 1991]).
- Advanced acceptance criteria take into account search history, e.g., by occasionally reverting to **incumbent solution**.

Examples

Example: Iterated Local Search for the TSP (1)

- **Given:** TSP instance π .
- **Search space:** Hamiltonian cycles in π .
- **Subsidiary local search:** Lin-Kernighan variable depth search algorithm
- **Perturbation mechanism:**
‘double-bridge move’ = particular 4-exchange step:



- **Acceptance criterion:** Always return the best of the two given candidate round trips.

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search**

- Variable Neighborhood Search

3. Population Based Metaheuristics

Tabu Search

Key idea: Avoid repeating history (memory)

How can we remember the history without

- memorizing full solutions (space)
- computing hash functions (time)

~> use attributes

Tabu Search

Key idea: Use aspects of search history (memory) to escape from local minima.

- Associate **tabu attributes** with candidate solutions or solution components.
- Forbid steps to search positions recently visited by underlying iterative best improvement procedure based on tabu attributes.

Tabu Search (TS):

determine initial candidate solution s

While **termination criterion** is not satisfied:

determine set N' of non-tabu neighbors of s

choose a best candidate solution s' in N'

update tabu attributes based on s'

$s := s'$

Example: Tabu Search for CSP

- **Search space:** set of all complete assignments of X .
- **Neighborhood structure:** one exchange
- **Memory:** Associate tabu status (Boolean value) with each pair (variable,value) (x, val) .
- **Initialization:** a random assignment
- **Search steps:**
 - pairs (x, v) are tabu if they have been changed in the last tt steps;
 - neighboring assignments are admissible if they can be reached by changing a non-tabu pair or have fewer unsatisfied constraints than the best assignments seen so far (**aspiration criterion**);
 - choose uniformly at random admissible neighbors with minimal number of unsatisfied constraints.
- **Termination:** upon finding a feasible assignment **or**
after given bound on number of search steps has been reached **or**
after a number of idle iterations

Note:

- **Admissible neighbors of s** : Non-tabu search positions in $N(s)$
- **Tabu tenure**: a fixed number of subsequent search steps for which the last search position or the solution components just added/removed from it are declared **tabu**
- **Aspiration criterion** (often used): specifies conditions under which tabu status may be overridden (*e.g.*, if considered step leads to improvement in incumbent solution).
- Crucial for efficient implementation:
 - efficient **best improvement** local search
 $\rightsquigarrow$ pruning, delta updates, (auxiliary) data structures
 - efficient determination of tabu status:
 store for each variable x the number of the search step when its value was last changed it_x ; x is tabu if $it - it_x < tt$, where it = current search step number.

Design Choices

Design choices:

- Neighborhood exploration:
 - no reduction
 - min-conflict heuristic
- Prohibition power for move = $\langle x, \text{new_v}, \text{old_v} \rangle$
 - $\langle x, -, - \rangle$
 - $\langle x, -, \text{old_v} \rangle$
 - $\langle x, \text{new_v}, \text{old_v} \rangle, \langle x, \text{old_v}, \text{new_v} \rangle$
- Tabu list dynamics:
 - Interval: $tt \in [t_b, t_b + w]$
 - Adaptive: $tt = \lfloor \alpha \cdot c \rfloor + \text{RandU}(0, t_b)$

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search**

3. Population Based Metaheuristics

Variable Neighborhood Search

Variable Neighborhood Search is a method based on the systematic change of the neighborhood during the search.

Central observations

- a local minimum w.r.t. one neighborhood function is not necessarily locally minimal w.r.t. another neighborhood function
- a global optimum is locally optimal w.r.t. **all** neighborhood functions

Key principle: change the neighborhood during the search

- Several adaptations of this central principle
 - (Basic) Variable Neighborhood Descent (VND)
 - Variable Neighborhood Search (VNS)
 - Reduced Variable Neighborhood Search (RVNS)
 - Variable Neighborhood Decomposition Search (VNDS)
 - Skewed Variable Neighborhood Search (SVNS)
- Notation
 - N_k , $k = 1, 2, \dots, k_m$ is a set of neighborhood functions
 - $N_k(s)$ is the set of solutions in the k -th neighborhood of s

How to generate the various neighborhood functions?

- for many problems different neighborhood functions (local searches) exist / are in use
- change parameters of existing local search algorithms
- use k -exchange neighborhoods; these can be naturally extended
- many neighborhood functions are associated with distance measures; in this case increase the distance

Basic Variable Neighborhood Descent

Procedure BVND

input : N_k , $k = 1, 2, \dots, k_{max}$, and an initial solution s

output: a local optimum s for N_k , $k = 1, 2, \dots, k_{max}$

$k \leftarrow 1$

repeat

$s' \leftarrow \text{FindBestNeighbor}(s, N_k)$

if $f(s') < f(s)$ **then**

$s \leftarrow s'$

$(k \leftarrow 1)$

else

$k \leftarrow k + 1$

until $k = k_{max}$;

Variable Neighborhood Descent

Procedure VND

input : N_k , $k = 1, 2, \dots, k_{max}$, and an initial solution s

output: a local optimum s for N_k , $k = 1, 2, \dots, k_{max}$

$k \leftarrow 1$

repeat

$s' \leftarrow \text{IterativeImprovement}(s, N_k)$

if $f(s') < f(s)$ **then**

$s \leftarrow s'$

$k \leftarrow 1$

else

$k \leftarrow k + 1$

until $k = k_{max}$;

- Final solution is locally optimal w.r.t. all neighborhoods
- First improvement may be applied instead of best improvement
- Typically, order neighborhoods from smallest to largest
- If iterative improvement algorithms $II_k, k = 1, \dots, k_{max}$ are available as black-box procedures:
 - order black-boxes
 - apply them in the given order
 - possibly iterate starting from the first one
 - order chosen by: **solution quality** and **speed**

Basic Variable Neighborhood Search

Procedure BVNS

input : N_k , $k = 1, 2, \dots, k_{max}$, and an initial solution s

output: a local optimum s for N_k , $k = 1, 2, \dots, k_{max}$

repeat

$k \leftarrow 1$

repeat

$s' \leftarrow \text{RandomPicking}(s, N_k)$

$s'' \leftarrow \text{IterativeImprovement}(s', N_k)$

if $f(s'') < f(s)$ **then**

$s \leftarrow s''$

$k \leftarrow 1$

else

$k \leftarrow k + 1$

until $k = k_{max}$;

until Termination Condition;

To decide:

- which neighborhoods
 - how many
 - which order
 - which change strategy
-
- Extended version: parameters k_{min} and k_{step} ; set $k \leftarrow k_{min}$ and increase by k_{step} if no better solution is found (achieves diversification)

Adaptive Large Neighborhood Search

<https://imada.sdu.dk/u/marco/DM841/slides/dm841-hr-alns.pdf>

Outline

1. Satisfiability

2. LS Based Metaheuristics

- Randomized (or Stochastic) Local Search

- Guided Local Search

- Simulated Annealing

- Iterated Local Search

- Tabu Search

- Variable Neighborhood Search

3. Population Based Metaheuristics

Ant Colony Optimization

<https://imada.sdu.dk/u/marco/DM841/slides/dm841-hr-aco.pdf>

Evolutionary Algorithms

<https://imada.sdu.dk/u/marco/DM841/slides/dm841-hr-evolutionary.pdf>

ROHs: Reasoning, Metaheuristics

- Stochastic Local Search
- Simulated Annealing
- Iterated Local Search
- Tabu Search
- Variable Neighborhood Search
- Adaptive Large Neighborhood Search
- Evolutionary Algorithms
- Ant Colony Optimization
- Estimation-of-Distribution Algorithms
- Artificial Immune Systems
- ...
- Evolutionary Computation Bestiary <http://fcampelo.github.io/EC-Bestiary/>
- **Super**natural inspired [Maturana, Fouhey, 2013]